

Documentação Técnica de Arquitetura de Software

Projeto: Robô Maze Rescue Junior (RoboCup Junior 2026)
Escopo: Arquitetura de software embarcado executado na Raspberry Pi, detalhando as responsabilidades, assinaturas de funções, gerenciamento de estado e os fluxos de comunicação inter-módulos.

1. Visão Geral do Sistema e Hierarquia de Controle

O sistema adota uma arquitetura hierárquica e desacoplada, dividida em três níveis principais de abstração:

- **Nível de Decisão (Alto Nível):** Implementado pelo módulo `dfs.py`, gerencia o mapeamento lógico do labirinto, controle de tempo (deadlines) e identificação de vítimas.
- **Nível de Coordenação e Controle de Baixo Nível:** Implementado por `navigation.py`, traduz comandos cardinais absolutos em malhas de controle de velocidade e direção, monitorando sensores em tempo real.
- **Nível de Driver e Abstração de Hardware:** Implementado por `serial_comm.py` e `sensor_floor.py`, lidando diretamente com a camada física de comunicação UART e periféricos de I/O.

2. Detalhamento dos Módulos

2.1. `config.py` — Fonte Única de Verdade (Single Source of Truth)

Este arquivo centraliza todas as parametrizações operacionais do sistema. A modificação dessas variáveis altera dinamicamente o comportamento cinemático e lógico do robô sem necessidade de refatoração de código.

Constante / Grupo	Descrição Técnica e Aplicação
Cardinais (NORTH, EAST, SOUTH, WEST)	Definição de inteiros (0, 1, 2, 3) para representar os quatro pontos cardinais, simplificando operações aritméticas de rotação com operador módulo.
Geometria do Labirinto	<code>CELL_DISTANCE_CM = 30.0</code> padroniza o tamanho físico de uma célula. <code>WALL_THRESHOLD_CM = 15.0</code> define a distância

	crítica para classificação de presença de parede.
Parâmetros Cinemáticos	MOTOR_SPEED = 40 (velocidade nominal) e RAMP_SPEED = 60 (torque/velocidade aumentada para vencer aclives).
Controle P da IMU	HEADING_KP = 0.5 ganho proporcional para correção de desvio de curso durante a translação retilínea.
Temporizadores de Missão	MISSION_TIMEOUT_S = 450 (7m30s) tempo limite total antes de forçar o algoritmo a abortar e retornar à origem (0,0).

2.2. serial_comm.py — Abstração de Comunicação UART

Esta classe gerencia a interface serial assíncrona entre o computador de bordo (Raspberry Pi) e o microcontrolador de execução (ESP32). Implementa um barramento robusto dotado de tolerância a falhas e emulação em software.

Componentes e Métodos Internos:

- **__init__(self, port, baudrate, simulate):** Inicializa o objeto serial. Caso simulate=True, o sistema desativa as chamadas ao driver de hardware da biblioteca pyserial e ativa o interpretador local de simulação, permitindo testes lógicos em ambiente PC.
- **ping(self, max_tries, interval) -> bool:** Executa o handshake inicial com o firmware enviando o comando "PG". Caso obtenha "OK" dentro do número de tentativas (max_tries), valida o canal. Caso contrário, retorna falso, disparando uma falha fatal no sistema.
- **_serial_send(self, command) -> str:** Invoca uma malha de repetição (retries) baseada na constante SERIAL_RETRIES. Se uma transação falhar por timeout ou ruído na linha, ela reinicia a tentativa após o tempo estipulado em SERIAL_RETRY_WAIT.
- **_raw_send(self, command) -> str:** Camada de manipulação de baixo nível. Limpa o buffer de recepção com reset_input_buffer(), anexa o terminador de linha \n, codifica em bytes via ASCII/UTF-8 e força a transmissão imediata via flush(). Aguarda a leitura da linha de resposta decodificando-a com tratamento de erro de substituição de caracteres.
- **_simulate_send(self, command) -> str:** Simula o comportamento físico do robô. Mantém variáveis internas de odometria emulada (_sim_enc, _sim_moving, _sim_reverse) que incrementam ou decrementam o contador de distância virtual simulando um atraso de 10ms a cada chamada do comando de leitura de encoders ("MR").

2.3. sensor_floor.py — Interface do Sensor de Cor

Este wrapper expõe de forma simplificada as leituras do sensor de cor TCS para detecção de anomalias no solo, com ênfase na identificação de superfícies pretas (ladrilhos proibidos) ou azuis (pontos de checkpoint/bônus).

Mecanismo de Degradação Graciosa (Graceful Degradation):

O construtor da classe encapsula a importação das bibliotecas `pigpio` e `sensor_cor` em um bloco `try-except`. Se o daemon do hardware não for encontrado (comum em desenvolvimento local no PC), o código captura a exceção, imprime um log de diagnóstico e muda para o modo **stub**, evitando que uma falha física de barramento I2C impeça a execução do restante da aplicação.

Métodos Principais:

- **is_preto(self) -> bool:** Executa a leitura no driver e valida se a string retornada é exatamente "preto". Retorna False em caso de exceção de hardware para evitar falsos positivos que aprisionariam o robô.
- **get_cor(self) -> str | None:** Retorna o rótulo cromático identificado do solo (ex: 'azul', 'verde', 'preto') usado na lógica do DFS.

2.4. navigation.py — Controle Cinemático e Malha Fechada

O módulo lida com a física do robô, manipulando desvios angulares e gerenciando a locomoção retilínea através de sensores de malha fechada (Unidade de Medição Inercial - IMU e Encoders das rodas).

Funções de Rotação (turn_to):

A função `turn_to(target_cardinal, serial, imu)` rotaciona o robô no próprio eixo coordenado até alinhar-se com o ângulo do cardinal alvo definido em `config.py`. Ela opera sob quatro regras de segurança:

1. **Desaceleração Proporcional:** Se a diferença angular calculada por `angle_diff()` for maior que a constante `TURN_SLOW_ZONE`, ela aplica velocidade máxima (`TURN_SPEED_FAST`). Ao entrar na zona de aproximação, reduz a velocidade para `TURN_SPEED_SLOW` visando conter a inércia mecânica.
2. **Filtro de Estabilização (Settled Cycles):** O robô só considera a rotação finalizada se permanecer por `TURN_SETTLED_CYCLES` leituras consecutivas dentro da margem de erro permitida por `TURN_TOLERANCE`.
3. **Deteção de Overshoot:** Se o robô ultrapassar o alvo sem amortecer a tempo na tolerância, o sinal matemático do erro inverte. O código detecta essa mudança de estado

instantaneamente e interrompe os motores para mitigar oscilações infinitas.

4. **Timeout de Proteção:** Limita a tentativa de giro ao tempo definido em `TURN_TIMEOUT`, evitando sobreaquecimento de motores caso o robô esteja preso fisicamente.

Funções de Translação (`move_forward`):

Comanda o avanço retilíneo exato de 30 cm (uma célula). O algoritmo executa o seguinte laço assíncrono de alta frequência:

- **Cálculo robusto de odometria (Leitura de Baseline):** Antes de acionar os motores, captura uma assinatura do estado dos contadores de pulso das rodas (`_read_encoder_baseline`). Durante a marcha, subtrai o valor atual desse baseline. A função `_robust_distance()` extrai a **mediana** dos quatro valores de deltas, descartando outliers gerados por rodas que sofreram derrapagem (slip).
- **Correção de Desvio Inercial (Heading Hold):** A cada 5 iterações (`HEADING_CORR_INTERVAL`), o desvio angular em relação ao norte absoluto é avaliado. Um controlador Proporcional simplificado calcula uma correção diferencial de velocidade: `corr = int(HEADING_KP * erro)`. Esse valor é somado aos motores de um lado e subtraído do outro, mantendo o robô em linha perfeitamente reta.
- **Tratamento de Ladrilho Preto:** Se `floor_sensor.is_preto()` retornar verdadeiro a qualquer momento do trajeto, a função aborta o avanço, corta a energia dos motores, chama o método interno `_reverse_by()` para recuar usando encoders até a posição segura anterior e retorna a flag "BLACK".
- **Deteção de Rampa:** Avalia a inclinação inercial obtida pelo giroscópio/acelerômetro. Se o ângulo decrescer além de `RAMP_ENTER_DEG`, delega o controle para a rotina `_traverse_ramp()`, que eleva a potência dos motores para `RAMP_SPEED` e sustenta o deslocamento até o plano normalizar acima de `RAMP_EXIT_DEG`.

2.5. dfs.py — Inteligência Lógica de Exploração

Este módulo implementa a inteligência algorítmica do robô através de uma busca em profundidade (Depth-First Search) modelada de forma iterativa, mitigando riscos de estouro de pilha de memória incondicional (Stack Overflow) comuns em implementações recursivas em sistemas embarcados.

Gerenciamento de Estado do Mapa Virtual:

- `stack`: Lista que armazena sequencialmente o caminho de coordenadas cartesianas (x, y) adotado pelo robô. Atua como o rastro lógico fundamental para o processo de retrocesso (backtracking).
- `visited`: Conjunto hash (Set) contendo as coordenadas físicas validadas e completamente

exploradas pelo robô.

- blocked: Conjunto de coordenadas mapeadas como ladrilhos pretos intransponíveis.
- cell_map: Dicionário estruturado no formato { (x,y): [direções_possíveis_restantes] }.

Ciclo de Execução do Laço Principal:

O laço roda enquanto houver nós pendentes na pilha. Ele divide-se em três etapas lógicas:

1. **Sensoriamento da Célula Nova (Fase A):** Caso a coordenada atual não conste em cell_map, o robô executa a leitura das paredes circundantes disparando comandos via _read_walls_reliable(). Ele faz o parse da string de resposta contendo as distâncias dos ultrassônicos esquerdo, frontal e direito, convertendo coordenadas relativas do chassi para orientações cardinais absolutas do mapa global. Ele também processa a câmera de visão computacional em busca de vítimas de cor ou texto, emitindo o sinalizador "VC" para despejar kits de salvamento. Por fim, monta as opções de movimento ordenadas por prioridade cinemática (Frente > Direita > Esquerda > Trás).
2. **Movimentação e Tentativa de Avanço (Fase B):** Retira a primeira opção disponível de caminho da célula atual. Se a célula de destino já foi visitada ou está marcada como bloqueada, descarta e passa para a próxima. Caso contrário, aciona o wrapper move_to_direction(). Se o retorno cinemático for "BLACK", anota o destino em blocked e permanece na célula original. Se foi "OK" ou "RAMP", valida a entrada incluindo o novo nó em visited e empilhando a coordenada em stack.
3. **Retrocesso Físico (Fase C / Backtracking):** Se a lista de opções de caminhos viáveis da célula atual for exaurida, significa que o robô atingiu um beco sem saída (Dead End) ou uma região totalmente cercada por nós visitados. O robô desempilha o nó atual (stack.pop()), lê qual era a coordenada imediatamente anterior na pilha, calcula a direção geométrica reversa através de direction_between() e aciona os motores para retornar fisicamente uma casa atrás.

Mecanismo de Retorno de Emergência: A cada iteração, o tempo corrente é confrontado com o parâmetro mission_deadline. Caso o prazo expire, o laço de busca é quebrado e o método especializado _return_to_start() assume o controle do robô, esvaziando a pilha sequencialmente em direção à coordenada de origem (0,0) para salvar os pontos acumulados da rodada antes do encerramento oficial da contagem de tempo.

2.6. main.py — Orquestrador e Inicializador do Ciclo de Vida

O ponto de entrada principal do programa gerencia os argumentos fornecidos via linha de comando através da biblioteca argparse, configura as instâncias de hardware globais e provê uma política rígida de encerramento seguro e contenção de danos mecânicos em caso de falha do software.

Toda a lógica central de setup é encapsulada em um bloco estrutural try-catch-finally. Independente do robô finalizar a exploração com sucesso, estourar o tempo limite de missão, sofrer uma interrupção manual por teclado (Ctrl+C), ou disparar uma exceção catastrófica não tratada no código (crash), o bloco finally possui prioridade absoluta de execução no processador, garantindo o envio imediato do comando serial "MC 0 0 0 0" para zerar a potência de todos os motores, além de encerrar os daemons do sensor de chão e os barramentos de streaming de captura da câmera (Picamera2).

3. Fluxo de Comunicação e Encapsulamento Inter-Módulos

O fluxo de dados obedece estritamente a uma topologia descendente de comando e ascendente de telemetria conforme detalhado a seguir:

Origem da Chamada	Módulo Destino	Dados Trafegados (Parâmetros)	Efeito Lógico / Retorno Esperado
main.py	dfs.py	Instâncias de barramento (Serial, IMU, Floor, Camera, Mission Deadline).	Inicia o laço de mapeamento e bloqueia a execução principal até a conclusão do labirinto.
dfs.py	serial_comm.py	Strings de comando padrão ("SR", "VC", "VICTIM...").	Solicita leitura direta dos sensores ultrassônicos anexados ao ESP32 ou sinaliza atuação de resgate.
dfs.py	navigation.py	Cardinal de destino desejado (inteiro de 0 a 3).	Chama o wrapper move_to_direction() que gerencia cinemática completa de movimento celular.
navigation.py	serial_comm.py	Strings cinemáticas ("MC v1 v2 v3 v4", "MR").	Aplica vetores de potência nas pontes H dos motores e

			requisita pulsos acumulados de odometria.
navigation.py	sensor_floor.py	Nenhum (Varredura de polling contínuo).	Invoca is_preto() para monitorar integridade física da célula durante o avanço mecânico.